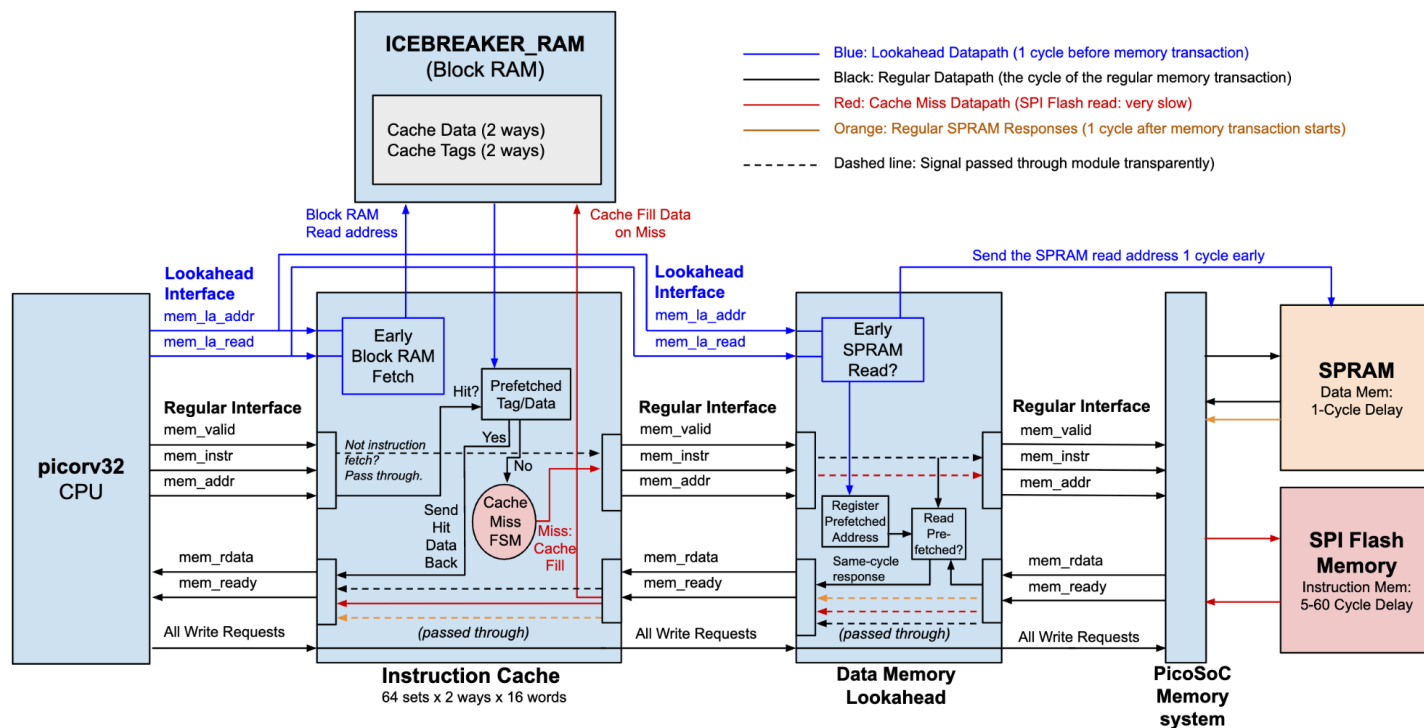


Memory System Design and Functionality

See the diagram below: it is also available in full size as Memory System Diagram.pdf

Diagram of Final Memory System with Instruction Cache and Data Memory Lookahead



Both the instruction cache and data memory lookahead module are designed to have interfaces matching the shape of the existing CPU interface, so memory transactions can easily flow through them. The one exception is the data memory lookahead module uses a separate side-interface to drive the SPRAM read address input for lookaheads. There is no architectural reason behind this other than the fact that it was found to improve timing (a bit counterintuitive...).

The modules are transparent to all requests that aren't related to it (e.g. all non-instruction-fetch requests for the instruction cache) and only intercepts relevant requests. This is shown in dotted lines on the diagram.

Instruction Cache

- Final version is 64 sets, 2 ways, and 16 words per way.
 - Uses FPGA BRAM (ICEBREAKER_RAM)
 - Associativity allows robustness to certain memory access patterns that cause tag collisions
 - More words per line exploits spatial locality (though this is maybe not a great argument because the SPI read stream seems to keep its spot so it doesn't incur a penalty for reading sequential instructions even if reads are separated in time). But most importantly it allows larger cache with the same logic cell usage (for things like MUXing, tag matching, etc). It comes at the cost of worse cache fill penalty.
 - Can store total 2048 instructions
 - Uses CPU lookahead interface to respond to memory transaction in the same cycle.

- Has a 3-state FSM for when it misses, to keep track of whether miss logic is idle, filling, or sending the response.
- In the RTL there are register-like signals called `la_cached_data` and `la_cached_tag`. However these are actually not synthesized as hardware registers, they are equivalent to (and implemented by) the registers at the BRAM synchronous output port. Writing it as a register in RTL seems to encourage better timing, though.

Data memory lookahead

- Uses CPU lookahead interface to set the read address for SPRAM 1 cycle early
- Therefore when the real request comes, the required data is already at the port and we can send it back immediately
- Also registers the RAM address that was lookaheaded the previous cycle.
 - Maybe not strictly necessary. But the formal assertions of the CPU don't guarantee that every read instruction was preceded by a lookahead so we do this just to be safe - allow a combinational tag comparison when the real memory read transaction comes through on the main interface. The cache has a similar check built in because tag is always checked.

Below we describe the dataflow in different scenarios through the memory system.

Any write

- Transaction is passed through transparently through the modules, both ways. So behaviour is exactly the same as before.

Cache Hit

- Cycle before (after n-1 rising edge): lookahead address stabilizes
- Rising edge of cycle n: synchronous BRAM read completes, and data/tag become available at the port of the RAM and also at signals called `la_cached_data` and `la_cached_tag` in the RTL (which should refer to the same hardware as the registers controlling the BRAM output port)
- Combinational tag check is performed during cycle n and the ready signal is thus asserted, and cached data is handed back
- Rising edge of cycle n+1: CPU sees valid && ready and completes the transaction (in the minimum possible time). No delay is incurred for the BRAM read due to use of lookahead.

Cache Miss

- Same as before until the combinational tag check, which fails
- FSM in cache advances to `S_FILL`, fill counter begins, filling all the words in the line with sequential SPI requests and everything gets written to the cache
- When done, state advances to `S_RESP`, cache delivers the response and CPU sees it. (Ensuring `RESP` is only sent until the whole `FILL` prevents further memory transactions initiated while cache is still busy). FSM only stays in this state for 1 cycle, then goes back to `IDLE`
- Transaction completes

Data read

GB3 Group 5 - Documentation and Results

- Cycle before (after n-1 rising edge): lookahead address stabilizes
- Data memory lookahead module sees this and tells SPRAM to load the address
- It also registers the value of the address being look-ahead to allow a correctness check (the register is not very big, only 8 bits)
- On cycle N, the register above latches the correct value, the real memory transaction comes through, combinational comparison is done, and if correct, the data in the SPRAM's port is directly sent back to CPU, skipping the delay entirely
- Rising edge of cycle N+1: transaction fully complete after CPU sees valid && ready.